

Programación Lógica

Fundamentos teóricos

(clase 3)

LPO – Sintaxis

Resumen

- Cláusulas $(A_1 \vee \dots \vee A_p \leftarrow B_1 \wedge \dots \wedge B_q)$
 - Cláusulas de Horn
 - Cláusulas definidas
 - reglas: $(A_1 \leftarrow B_1 \wedge \dots \wedge B_q)$
 - hechos: $(A_1 \leftarrow)$
 - Objetivos definidos $(\leftarrow B_1 \wedge \dots \wedge B_q)$

(Recordar: Los A_i y B_i son literales. Asumimos la clausura universal.)

Pruebas en prolog

Para probar que una consulta C es consecuencia lógica de un programa lógico P basta con probar que $P \cup \{\neg C\}$ es insatisfactible.

A este tipo de demostración se le llama **prueba por refutación**: se niega lo que se quiere probar, llegando a un absurdo.

Debemos buscar una forma automática de resolver este problema, para esto se introducen los **métodos de resolución**.

Sustituciones

Si tenemos variables libres podemos sustituirlas por términos cualesquiera.

Definición:

Una sustitución θ es un conjunto finito de la forma $\{v_1/t_1, \dots, v_n/t_n\}$ en donde, para $1 \leq i \leq n$:

- v_i es una variable
- t_i es un término distinto de v_i
- $v_i \neq v_j$ si $i \neq j$

Ejemplos

¿Son sustituciones los siguientes conjuntos?

$\{x/1, y/f(h,x)\}$

$\{x/f(y), z/z\}$

$\{x/1, y/f(h,x), x/2\}$

Instancia

Def. Instancia de una fórmula según una sustitución

Sea E una expresión (término o fórmula sin cuantificadores),
y $\theta = \{v_1/t_1, \dots, v_n/t_n\}$ una sustitución :

$E\theta$, **instancia** de E por θ , es el resultado de la aplicación **simultánea** del reemplazo de cada variable v_i por el término t_i en E .

La noción de instancia se extiende a **conjuntos** de fórmulas:

Si $S = \{E_1, \dots, E_n\}$ es un conjunto de expresiones y θ una sustitución,
 $S\theta$, **instancia del conjunto** S por θ , es el conjunto $\{E_1\theta, \dots, E_n\theta\}$

Ejemplo

Si $E = p(x, y, f(b, x, z))$ y $\theta = \{x/a, y/x, z/f(z)\}$

$E\theta = ?$

Composición de sustituciones

Def. Composición de sustituciones

Sean $\sigma = \{v_1/s_1, \dots, v_n/s_n\}$ y $\theta = \{u_1/t_1, \dots, u_m/t_m\}$ dos sustituciones :
la **composición $\sigma \circ \theta$ de σ y θ** es la sustitución que resulta de

$\{v_1/s_1\theta, \dots, v_n/s_n\theta, u_1/t_1, \dots, u_m/t_m\}$

luego de eliminar:

- los pares $v_i/s_i\theta$ t.q. $v_i = s_i\theta$
- los pares u_j/t_j tales que $u_j \in \{v_1, \dots, v_n\}$

Ejemplo:

$\theta_1 = \{x|a, y|f(z), z|y\}$ y $\theta_2 = \{x|b, y|z, z|g(x)\}$ sustituciones.

$\theta_1 \circ \theta_2 = ?$

Unificación

Def. Unificador

Sea S un conjunto de expresiones.

Una sustitución θ es un unificador de S si $S\theta$ es un conjunto con un único elemento (*singleton*).

(Dicho de otro modo:

θ es unificador de S si para todas $E_i, E_j \in S$, $E_i\theta \equiv E_j\theta$)

Ejemplo:

$S = \{p(x, y, z), p(a, f(x), z)\}$

$\theta = \{x/a, y/f(a), z/b\}$ es un unificador, $S\theta = ?$

Unificador más general

Def. Unificador más general (m.g.u.)

θ es un mgu para $E1$ y $E2$ si:

- a. θ es un unificador para $E1$ y $E2$ y
- b. si σ es un unificador para $E1$ y $E2$, existe una sustitución γ t.q. $\theta \circ \gamma = \sigma$

El mgu de dos expresiones no tiene por qué ser único, p.ej., tanto $\theta1 = \{x/y\}$ como $\theta2 = \{y/x\}$ son mgus para $p(x)$ y $p(y)$.

Algoritmo de unificación

Entrada: E1 y E2

Salida: μ , mgu de las expresiones, o *no unifican* si E1 y E2 no son unificables.

Inicialización: PUSH E1 = E2; $\mu = \varepsilon$;

Mientras el stack no se vacíe

POP X=Y

En caso de que

1. X es una variable que no ocurre en Y:

$\mu = \mu \circ \{X/Y\}$;

sustituir Y por X en el stack, o sea, stack = stack $\circ \{X/Y\}$;

2. Y es una variable que no ocurre en X:

$\mu = \mu \circ \{Y/X\}$; stack = stack $\circ \{Y/X\}$;

3. X e Y son el mismo término: skip

4. X es $f(X_1, \dots, X_n)$, Y es $f(Y_1, \dots, Y_n)$:

PUSH $X_i = Y_i$; $i = 1 :: n$.

5. en otro caso: retornar *no unifican*.

Fin mientras

Retornar μ

Ejercicio

Práctico 4, ejercicio 6, parte b

Calcular el mgu de

$c(a, X, f(g(Y)))$ y $c(Z, h(Z, W), f(W))$

Resolución en cláusulas de 1er orden

Def.

Para $C1$ y $C2$ cláusulas de 1er orden, **sin variables en común**

$C1: \{L1, L2, \dots, Ln\}$

$C2: \{M1, M2, \dots, Mh\}$

θ , mgu de Li y $\neg Mj$

resolvente $C3$:

$\{L1, L2, \dots, Li-1, Li+1, \dots, Ln, M1, \dots, Mj-1, Mj+1, \dots, Mh\} \theta$

Resolución para cláusulas de 1er orden

Correctitud

Sean $C1$ y $C2$ dos cláusulas de 1er orden. Si R es una resolvente de $C1$ y $C2$ se cumple:

$$C1, C2 \models R$$

Resolución para cláusulas de 1er orden

Completitud

$S \models C$ implica $S \vdash_R C$?

No podemos derivar $p(f(y))$ a partir de $p(x)$, aunque
 $p(x) \models p(f(y))$

Sí podemos probar
 $\{p(x), \neg p(f(y))\}$ es inconsistente.

Procedimientos de resolución

Aplicación sistemática de resolución

- Saturación por niveles (completa)
- Resolución lineal (completa)
- Resolución con cláusula de entrada (no es completa)

(¡Estudiar todos los métodos en materiales publicados!)

Resolución SLD

- La resolución con cláusula de entrada es completa para cláusulas de Horn (cláusulas definidas + objetivo definido).
- Resolución SLD (Selection Linear Definite) es un tipo particular de **resolución lineal con cláusulas de entrada** para conjuntos de cláusulas de Horn.
- Siempre se parte del objetivo, por lo que en todos los pasos las resolventes también son objetivos. ¿Por qué?

Resolución SLD

Def. Resolvente-SLD.

Sea P un programa definido, G un objetivo, $\leftarrow A_1, \dots, A_n$.

G' es la **resolvente SLD** de G y una cláusula $A \leftarrow B_1, \dots, B_n$ de P con mgu θ si:

- A_m es un átomo (átomo seleccionado) de G
- θ es un mgu de A_m y A
- G' es $\leftarrow (A_1, \dots, A_{m-1}, B_1, \dots, B_n, A_{m+1}, \dots, A_n)\theta$

Derivación SLD

Sea P un programa definido, G un objetivo definido $\leftarrow A_1, \dots, A_n$.

Una **derivación-SLD** de $P \cup \{G\}$ es:

- una secuencia $G_0=G, G_1, G_2, \dots$ de objetivos,
- una secuencia $C_1, C_2, \dots$ de variantes de cláusulas de P y
- una secuencia $\theta_1, \theta_2, \dots$ de mgus tales que cada G_{i+1} se deriva de G_i y C_i con unificador θ_i .

Refutación SLD

Una **refutación-SLD** para $P \cup \{G\}$ es una derivación-SLD finita cuyo último objetivo es $\square$.

Procedimientos de refutación-SLD.

- El objetivo de un intérprete es, dado un programa P y una consulta C , encontrar una refutación-SLD para $P \cup \{G\}$, siendo $G = \neg C$.
- Una refutación está compuesta por una cantidad finita de pasos de resolución-SLD. En cada uno de estos pasos hay 2 aspectos que podrían resolverse de más de un modo:
 - cuál es el átomo del objetivo que se selecciona para reducir.
 - cuál es la cláusula del programa que reduce con el átomo seleccionado del objetivo.

Respuesta computada / Respuesta correcta

Def. Respuesta computada.

Una respuesta computada para $P \cup \{G\}$ es la sustitución obtenida restringiendo a las variables de G la composición de mgus utilizados en una refutación-SLD de $P \cup \{G\}$.

Def. Respuesta correcta

Sean P un programa lógico, G un objetivo, θ una sustitución para las variables de G , θ es una respuesta correcta si

$\forall x_1 \dots \forall x_k ((A_1, A_2, \dots, A_k)\theta)$ es consecuencia lógica de P .

Completitud de resolución-SLD

Para cláusulas de Horn la resolución-SLD es completa.

En otros términos, si $P \cup \{G\}$ es insatisfactible, existe una refutación-SLD para $P \cup \{G\}$.

Completitud respecto a respuestas

Si θ es una respuesta correcta, existe una respuesta computada σ y una sustitución γ tales que $\theta = \sigma\gamma$.

- Notar que las respuestas correctas no son necesariamente las más generales.

Árbol-SLD

Regla de computación:

Política de selección del siguiente átomo a reducir en un objetivo.

La regla de computación no afecta la existencia de una refutación-SLD.

Considerando una regla de computación fija, el espacio de búsqueda (asociado a las distintas formas de elegir las cláusulas del programa) es un árbol.

Lo llamamos **árbol-SLD**.

Árbol-SLD

Def. Sea P un programa lógico, G un objetivo.

Un **árbol-SLD para $P \cup \{G\}$** es un árbol que satisface:

1. Todo nodo es un objetivo (eventualmente vacío).
2. La raíz es G .
3. Sea $\leftarrow A_1, \dots, A_m, \dots, A_k$, $k > 0$, un nodo del árbol. Supongamos que A_m es el átomo seleccionado. Entonces, para toda cláusula $A \leftarrow B_1, \dots, B_q$ tal que A_m y A unifican con mgu θ , hay un hijo cuyo objetivo es la resolvente- SLD entre el objetivo y la cláusula: $\leftarrow (A_1, \dots, A_{m-1}, B_1, \dots, B_q, A_{m+1}, \dots, A_k)\theta$
4. Los nodos con la cláusula vacía no tienen hijos

Árbol-SLD

- Cada rama del árbol SLD corresponde a una posible derivación SLD.
- Por lo tanto, el árbol SLD representa todas las posibles derivaciones SLD a partir de un objetivo, utilizando una determinada regla de computación.
- La regla de computación determina las ramas del árbol: para un mismo programa lógico y objetivo, el árbol SLD posiblemente cambie al cambiar la regla de computación.
- A las ramas que corresponden a refutaciones (la hoja es un objetivo vacío) se les llama **ramas exitosas**.
- A las ramas que corresponden a derivaciones infinitas se les llama **ramas infinitas**.
- A las ramas que corresponden a derivaciones fallidas se les llama **ramas de falla**.

Refutación SLD

Independencia de la regla de computación

Sea P un programa lógico, G un objetivo :

Si existe una refutación-SLD para $P \cup \{G\}$ con respuesta computada θ , entonces para cualquier regla de computación R existe una refutación-SLD con respuesta computada θ' , tal que $G\theta'$ es una variante de $G\theta$.

Procedimiento de refutación SLD

Definición

Una regla de computación junto a una estrategia de búsqueda en un árbol SLD definen un **procedimiento de refutación SLD**.

Dos estrategias clásicas: BFS y DFS.

- Si el árbol es finito, ambas llegan a todos los nodos del árbol (posiblemente en distinto orden).
- Si no, BFS sigue llegando a cualquier nodo en una cantidad finita de pasos pero DFS no (aunque es más eficiente).

Procedimiento de refutación SLD

Un intérprete Prolog estándar utiliza el siguiente procedimiento de refutación SLD:

- regla de computación: elegir el átomo de más a la izquierda.
- estrategia de búsqueda: en profundidad, eligiendo las ramas del árbol según el orden de aparición de las cláusulas en el programa.

Por lo tanto, Prolog es incompleto: existen objetivos para los cuales hay alguna refutación SLD que el intérprete es incapaz de encontrar.

Ejercicio

Práctico 5, ejercicio 1

Esta presentación es solo un apoyo para el trabajo en clase.

Se debe estudiar el material teórico completo (está publicado en eva).

